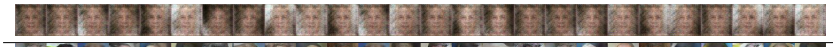
random sample	MMD	KNN	NN
	0.158	0.830	0.999
	0.154	0.994	1.000
	0.048	0.962	1.000
	**0.012**	0.798	0.964
	0.024	0.748	**0.949**
	0.019	**0.670**	0.983
	0.152	0.940	1.000
	0.222	0.978	1.000
	0.715	1.000	1.000
	**0.015**	0.817	0.987
	0.020	0.784	**0.950**
	0.024	**0.697**	0.971

Table 3: Results on GAN evaluation. Lower test statistics are best. Full results in Appendix A.

where $d(x)$ depicts the probability of the example x following the data distribution $P(X)$ versus being synthesized by the generator. This is according to a trainable *discriminator* function d. In the adversarial game, the generator g plays to fool the discriminator d by transforming noise vectors $z \sim P(Z)$ into real-looking examples $g(z)$. On the opposite side, the discriminator plays to distinguish between real examples x and synthesized examples $g(z)$. To approximate the solution to (3), alternate the optimization of the two losses (Goodfellow et al., 2014) given by

$$\begin{aligned} L_d(d) &= \mathbb{E}_x \left[\ell(d(x), 1)\right] + \mathbb{E}_z \left[\ell(d(g(z)), 0)\right], \\ L_g(g) &= \mathbb{E}_x \left[\ell(d(x), 0)\right] + \mathbb{E}_z \left[\ell(d(g(z)), 1)\right]. \end{aligned} \tag{4}$$

Under the formalization (4), the adversarial game reduces to the sequential minimization of $L_d(d)$ and $L_g(g)$, and reveals the true goal of the discriminator: to be the C2ST that best distinguishes data examples $x \sim P$ and synthesized examples $\hat{x} \sim \hat{P}$, where $\hat{P}$ is the probability distribution induced by sampling $z \sim P(Z)$ and computing $\hat{x} = g(z)$. The formalization (4) unveils the existence of an arbitrary binary classification loss function ℓ (See Remark 2), which in turn decides the divergence minimized between the real and fake data distributions (Nowozin et al., 2016).

Unfortunately, the evaluation of the log-likelihood of a GANs is intractable. Therefore, we will employ a two-sample test to evaluate the quality of the fake examples $\hat{x} = g(z)$. In simple terms, evaluating a GAN in this manner amounts to withhold some real data from the training process, and use it later in a two-sample test against the same amount of synthesized data. When the two-sample test is a binary classifier (as discussed in Section 3), this procedure is simply *training a fresh discriminator on a fresh set of data*. Since we train and test this *fresh* discriminator on held-out examples, it may differ from the discriminator trained along the GAN. In particular, the discriminator trained along with the GAN may have over-fitted to particular artifacts produced by the generator, thus becoming a poor C2ST.

We evaluate the use of two-sample tests for model selection in GANs. To this end, we train a number of DCGANs (Radford et al., 2016) on the bedroom class of LSUN (Yu et al., 2015) and the Labeled Faces in the Wild (LFW) dataset (Huang et al., 2007). We reused the Torch7 code of Radford et al. (2016) to train a set of DCGANs for $\{1, 10, 50, 100, 200\}$ epochs, where the generator and discriminator networks are convolutional neural networks (LeCun et al., 1998) with $\{1, 2, 4, 8\} \times$ gf and $\{1, 2, 4, 8\} \times$ df filters per layer, respectively. We evaluate each DCGAN on $10,000$ held-out examples using the fastest multi-dimensional two-sample tests: MMD, C2ST-NN, and C2ST-KNN.

Our first experiments revealed an interesting result. When performing two-sample tests directly on pixels, all tests obtain near-perfect test accuracy when distinguishing between real and synthesized (fake) examples. Such near-perfect accuracy happens consistently across DCGANs, regardless of the visual quality of their examples. This is because, albeit visually appealing, the fake examples contain checkerboard-like artifacts that are sufficient for the tests to consistently differentiate between real and fake examples. Odena et al. (2016) discovered this phenomenon concurrently with us.

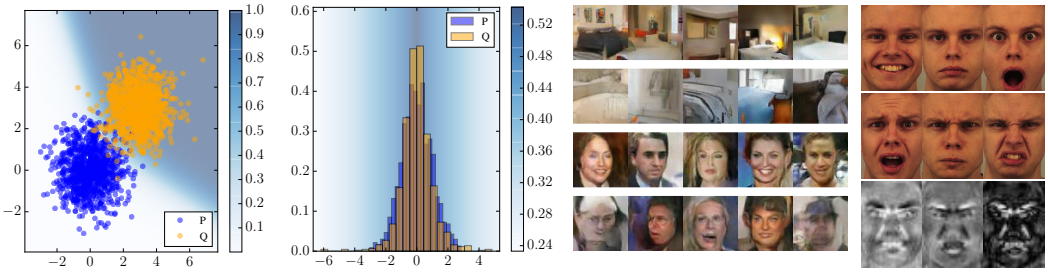


Figure 2: Interpretability of C2ST. The color map corresponds to the value of $p(l = 1|z)$.

On a second series of experiments, we featurize all images (both real and fake) using a deep convolutional ResNet (He et al., 2015) pre-trained on ImageNet, a large dataset of natural images (Russakovsky et al., 2015). In particular, we use the `resnet-34` model from Gross & Wilber (2016). Reusing a model pre-trained on natural images ensures that the test will distinguish between real and fake examples based only on natural image statistics, such as Gabor filters, edge detectors, and so on. Such a strategy is similar to perceptual losses (Johnson et al., 2016) and inception scores (Salimans et al., 2016). In short, in order to evaluate how natural the images synthesized by a DCGAN look, one must employ a "natural discriminator". Table 3 shows three GANs producing poor samples and three GANs producing good samples for the LSUN and LFW datasets, according to the MMD, C2ST-KNN, C2ST-NN tests on top of ResNet features. See Appendix A for the full list of results. Although it is challenging to provide with an objective evaluation of our results, we believe that the rankings provided by two-sample tests could serve for efficient early stopping and model selection.

Remark 3 (How good is my GAN? Is it overfitting?)**.** *Evaluating generative models is a delicate issue (Theis et al., 2016), but two-sample tests may offer some guidance. In particular, good (non-overfitting) generative models should produce similar two-sample test statistics when comparing their generated samples to both the train-set and the test-set samples.* [3] *As a general recipe, prefer* the smallest *(in number of parameters) generative model that achieves the* same and small *two-sample test statistic when comparing their generated samples to both the train-set and test-set samples.*

We have seen that GANs of different quality may lead to the same (perfect) C2ST statistic. To allow a finer comparison between generative models, we recommend implementing C2ST using a margin classifier with finite norm, or using as statistic the whole area under the C2ST training curve (on train-set or test-set samples).

5.1 Experiments on Interpretability

We illustrate the interpretability power of C2ST. First, the predictive uncertainty of C2ST sheds light on where the two samples under consideration agree or differ. In the context of GANs, this interpretability is useful to locate captured or dropped modes. In the first plot of Figure 2, a C2ST-NN separates two bivariate Gaussian distributions with different means. When performing this separation, the C2ST-NN provides an explicit decision boundary that illustrates *where* the two distributions separate from each other. In the second plot of Figure 2, a C2ST-NN separates a Gaussian distribution from a Student's t distribution with $\nu = 3$, after scaling both to zero-mean and unit-variance. The plot reveals that the peaks of the distributions are their most differentiating feature. Finally, the third plot of Figure 2 displays, for the LFW and LSUN datasets, five examples classified as real with high uncertainty (first row, better looking examples), and five examples classified as fake with high certainty (second row, worse looking examples).

Second, the features learnt by the classifier of a C2ST are also a mechanism to understand the differences between the two samples under study. The third plot of Figure 2 shows six examples from the Karolinska Directed Emotional Faces dataset, analyzed in Section 4.1. In that same figure, we arrange the weights of the first linear layer of C2ST-NN into the feature most activated at positive examples (bottom left, positive facial expressions), the feature most activated at negative

[3] As discussed with Arthur Gretton, if the generative model memorizes the train-set samples, a sufficiently large set of generated samples would reveal such memorization to the two-sample test. This is because some unique samples would appear multiple times in the set of generated samples, but not in the test-set of samples.

examples (bottom middle, negative facial expressions), and the "discriminative feature", obtained by substracting these two features (bottom right). The discriminative feature of C2ST-NN agrees with the one found by (Jitkrittum et al., 2016): positive and negative facial expressions are best distinguished at the eyebrows, smile lines, and lips. A similar analysis Jitkrittum et al. (2016) on the C2ST-NN features in the NIPS article classification problem (Section 4.1) reveals that the features most activated for the "statistical learning theory" category are those associated to the words *inequ*, *tight*, *power*, *sign*, *hypothesi*, *norm*, *hilbert*. The features most activated for the "Bayesian inference" category are those associated to the words *infer*, *markov*, *graphic*, *conjug*, *carlo*, *automat*, *laplac*.

6 Experiments on Conditional GANs for Causal Discovery

In causal discovery, we study the causal structure underlying a set of d random variables $X_1, \ldots, X_d$. In particular, we assume that the random variables $X_1, \ldots, X_d$ share a causal structure described by a collection of Structural Equations, or SEs (Pearl, 2009). More specifically, we assume that the random variable X_i takes values as described by the SE $X_i = g_i(\text{Pa}(X_i, \mathcal{G}), N_i)$, for all $i = 1, \ldots, d$. In the previous, $\mathcal{G}$ is a Directed Acyclic Graph (DAG) with vertices associated to each of the random variables $X_1, \ldots, X_d$. Also in the same equation, $\text{Pa}(X_i, \mathcal{G})$ denotes the set of random variables which are parents of X_i in the graph $\mathcal{G}$, and N_i is an independent noise random variable that follows the probability distribution $P(N_i)$. Then, we say that $X_i \to X_j$ if $X_i \in \text{Pa}(X_j)$, since a change in X_i will *cause* a change in X_j, as described by the i-th SE.

The goal of causal discovery is to infer the causal graph $\mathcal{G}$ given a sample from $P(X_1, \ldots, X_d)$. For the sake of simplicity, we focus on the discovery of causal relations between two random variables, denoted by X and Y. That is, given the sample $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n \sim P^n(X, Y)$, our goal is to conclude whether "X causes Y", or "Y causes X". We call this problem *cause-effect discovery* (Mooij et al., 2016). In the case where $X \to Y$, we can write the cause-effect relationship as:

$$x \sim P(X), \quad n \sim P(N), \quad y \leftarrow g(x, n). \tag{5}$$

The current state-of-the-art in the cause-effect discovery is the family of Additive Noise Models, or ANM (Mooij et al., 2016). These methods assume that the SE (5) allow the expression $y \leftarrow g(x) + n$, and exploit the independence assumption between the cause random variable X and the noise random variable N to analyze the distribution of nonlinear regression residuals, in both causal directions.

Unfortunately, assuming independent additive noise is often too simplistic (for instance, the noise could be heteroskedastic or multiplicative). Because of this reason, we propose to use Conditional Generative Adversarial Networks, or CGANs (Mirza & Osindero, 2014) to address the problem of cause-effect discovery. Our motivation is the shocking resemblance between the generator of a CGAN and the SE (5): the random variable X is the conditioning variable input to the generator, the random variable N is the noise variable input to the generator, and the random variable Y is the variable synthesized by the generator. Furthermore, CGANs respect the independence between the cause X and the noise N by construction, since $n \sim P(N)$ is independent from all other variables. This way, CGANs bypass the additive noise assumption naturally, and allow arbitrary interactions $g(X, N)$ between the cause variable X and the noise variable N.

To implement our cause-effect inference algorithm in practice, recall that training a CGAN from X to Y minimizes the two following objectives in alternation:

$$L_d(d) = \mathbb{E}_{x,y}\left[\ell(d(x, y), 1)\right] + \mathbb{E}_{x,z}\left[\ell(d(x, g(x, z)), 0)\right],$$
$$L_g(g) = \mathbb{E}_{x,y}\left[\ell(d(x, y), 0)\right] + \mathbb{E}_{x,z}\left[\ell(d(x, g(x, z)), 1)\right].$$

Our recipe for cause-effect is to learn two CGANs: one with a generator g_y from X to Y to synthesize the dataset $\mathcal{D}_{X \to Y} = \{(x_i, g_y(x_i, z_i))\}_{i=1}^n$, and one with a generator g_x from Y to X to synthesize the dataset $\mathcal{D}_{X \leftarrow Y} = \{(g_x(y_i, z_i), y_i)\}_{i=1}^n$. Then, we prefer the causal direction $X \to Y$ if the two-sample test statistic between the real sample $\mathcal{D}$ and $\mathcal{D}_{X \to Y}$ is smaller than the one between $\mathcal{D}$ and $\mathcal{D}_{Y \to X}$. Thus, our method is Occam's razor at play: declare the simplest direction (in terms of conditional generative modeling) as the true causal direction.

Table 4 summarizes the performance of this procedure when applied to the 99 Tübingen cause-effect pairs dataset, version August 2016 (Mooij et al., 2016). RCC is the Randomized Causation Coefficient of (Lopez-Paz et al., 2015). The Ensemble-CGAN-C2ST trains 100 CGANs, and decides the causal direction by comparing the top generator obtained in each causal direction, as told by C2ST-KNN.